

FIXPOINT SWARM-R

Phase 4: DSHAE Integration, Desktop GUI & Validation Sandbox

Build Specification for Coding Agent

Input: Phase 3 repository (16 crates, 176 tests, TUI, persistence, Binance+Kraken, TTCP, sniper mode)

Additions: DSHAE Dual-Simplex engine (integrated),
egui desktop GUI with visual dashboard,
offline validation sandbox with bundled data

New crates: `fsr-dshae` (Dual-Simplex + HIM + Crystal),
`fsr-gui` (egui desktop application)

Total crates: 18 (16 existing + 2 new)

Date: 12 March 2026

Prime Directive

Phase 4 has two goals: (1) integrate the DSHAE Dual-Simplex arbitrage engine as the specialized signal source for triangle/multi-leg mispricing detection, and (2) give the operator a real desktop GUI with charts, visual feedback, and a one-click sandbox mode that lets a near-layperson test and validate the system entirely offline. After Phase 4, the system is a complete desktop trading instrument with a graphical interface.

Contents

1	Phase 4 Scope	2
1.1	What Phase 4 Adds	2
1.2	What Phase 4 Does NOT Add	2
2	DSHAE Integration	2
2.1	Integration Mode	2
2.2	New Crate: fsr-dshae	3
2.2.1	Currency Basket	3
2.2.2	Dual Simplex	3
2.2.3	Holographic Interference Manifold	4
2.2.4	Crystal Formation	4
2.3	DSHAE Configuration	5
3	Desktop GUI	5
3.1	Technology	5
3.2	New Crate: fsr-gui	5
3.3	GUI Layout	6
3.4	Tab Descriptions	6
3.5	Engine-to-GUI Communication	7
3.6	Launch	7
4	Validation Sandbox	7
4.1	Bundled Test Data	8
4.2	Synthetic Data Generator	8
4.3	Sandbox GUI Tab	9
4.4	Scenario Validation Logic	9
5	Module Placement	10
6	New Dependencies	10
7	Implementation Order	11
8	Definition of Done (Phase 4)	12
9	Phase 5 Preview (Non-Normative)	12

1 Phase 4 Scope

1.1 What Phase 4 Adds

1. **DSHAE engine** (`fsr-dshae`): Dual-Simplex observation, Cross-Rate Tensor, Deviation Tensor, Holographic Interference Manifold (HIM) construction, Crystal formation. Operates in **integrated** mode: replaces the raw candidate discovery in L1/L2 trumpet layers with structurally validated, market-neutral arbitrage signals.
2. **Desktop GUI** (`fsr-gui`): native desktop application via `egui/eframe`. Replaces TUI as the primary operator interface (TUI remains as fallback for SSH/headless). Visual charts for P&L, resonance metrics, regime history, crystal events, TTCP convergence, and the HIM point cloud.
3. **Validation Sandbox**: a one-click mode that runs a full backtest on bundled synthetic data, producing a visual report in the GUI. No configuration required beyond launching the app. Designed for someone who has never seen the system before.
4. **Bundled Test Data**: pre-generated synthetic market recordings (with known injected arbitrage opportunities and known no-arbitrage periods) shipped in `fixtures/sandbox/`. These are the “known answer” datasets for empirical validation.

1.2 What Phase 4 Does NOT Add

- No web interface (desktop-native only via `egui`).
- No new venue adapters beyond Binance + Kraken.
- No derivatives or margin trading.
- No ML components.

2 DSHAE Integration

2.1 Integration Mode

DSHAE runs in **integrated** mode (DSHAE Spec §17): it replaces the raw trumpet L1/L2 candidate discovery with its own Dual-Simplex → HIM → Cascade → Crystal pipeline. The existing FSR gate stack, CSP execution, hedge, and evidence chain remain unchanged. DSHAE is a smarter signal source, not a replacement for the execution framework.

Listing 1: Integration point in macro-cycle

```
// In fsr-runtime macro-cycle, step 8 (expand_trumpet):
if config.dshae.enabled {
    // DSHAE replaces raw trumpet expansion with Crystal-based signals
    let him = dshae_engine.construct_him(&market_snapshot, tick);
    let crystals = dshae_engine.run_cascade(&him, &config.dshae.cascade);
    candidates = crystals.iter()
        .filter(|c| c.net_edge >= config.arb.tau_edge_bps)
        .map(|c| c.to_candidate_cycle())
        .collect();
} else {
    // Original trumpet expansion (L1/L2/L3/L4)
    candidates = trumpet_expand(&market_snapshot, &config.trumpet);
}
// From here: same gate stack, same CSP, same execution
```

2.2 New Crate: fsr-dshae

Listing 2: fsr-dshae module structure

```
fsr-dshae/
src/
  lib.rs           // public API
  basket.rs        // Currency basket management
  simplex.rs       // Alpha/Beta cycle computation
  tensor.rs        // Cross-Rate Tensor R(t), Deviation Tensor D(t)
  him.rs           // Holographic Interference Manifold construction
  crystal.rs       // Crystal formation and validation
  axle.rs          // Axle-Invariant enforcement
  config.rs        // DSHAE config structs
```

2.2.1 Currency Basket

Listing 3: Basket and tensor types

```
pub struct CurrencyBasket {
  pub currencies: Vec<CurrencyId>, // e.g., ["BTC","ETH","SOL","USDT"]
  pub n: usize,                    // basket size
}

pub struct CrossRateTensor {
  pub rates: Vec<Vec<Q32>>>,      // N x N matrix, rates[i][j] = r_ij
  pub tick: u64,
}

pub struct DeviationTensor {
  pub deviations: Vec<Vec<Vec<Q32>>>>, // N x N x N, D[i][j][k]
  pub tick: u64,
}

impl CrossRateTensor {
  pub fn deviation_tensor(&self) -> DeviationTensor {
    // D_ijk = log(r_ij) + log(r_jk) - log(r_ik)
    // D_ijk = 0 means no-arbitrage; D_ijk != 0 means mispricing
  }
}
```

2.2.2 Dual Simplex

Listing 4: Alpha/Beta simplex

```
pub struct DualSimplex {
  pub basket: CurrencyBasket,
  pub alpha_cycle: Vec<usize>, // forward cycle indices
  pub beta_cycle: Vec<usize>,  // reverse cycle indices
  pub phi_alpha: Q32,          // phase
  pub phi_beta: Q32,           // anti-phase (phi + pi)
}

impl DualSimplex {
  pub fn tick(&mut self, rates: &CrossRateTensor) -> SimplexObservation {
    let p_alpha = self.compute_cycle_product(rates, &self.alpha_cycle);
    let p_beta = self.compute_cycle_product(rates, &self.beta_cycle);
  }
}
```

```

    // Axle invariant: net exposure = 0
    assert!(self.check_axle_invariant());
    SimplexObservation { p_alpha, p_beta, phase_diff: /*...*/ }
}
}

```

2.2.3 Holographic Interference Manifold

Listing 5: HIM construction

```

pub struct HimPoint {
    pub x1: Q32,    // log(r_ij)
    pub x2: Q32,    // log(r_jk)
    pub x3: Q32,    // D_ijk (deviation)
    pub psi: Q32,   // |delta_ij| + |delta_jk| (phase differential)
    pub omega: Q32, // phi(w) mod 2pi
    pub triangle: (usize, usize, usize), // (i, j, k) indices
}

pub struct Him {
    pub points: Vec<HimPoint>, // |T(B)| = N*(N-1)*(N-2)/6 points
    pub tick: u64,
    pub window: u64,
    pub digest: Hash256,
}

impl Him {
    pub fn construct(
        basket: &CurrencyBasket,
        rates: &CrossRateTensor,
        deviations: &DeviationTensor,
        simplex: &DualSimplex,
        phase: Q32,
    ) -> Self {
        // For every triangle (i,j,k) in T(B):
        //   sigma_ijk = (log(r_ij), log(r_jk), D_ijk, |delta_ij|+|delta_jk|, phi)
    }
}

```

2.2.4 Crystal Formation

Listing 6: Crystal from DSHAE cascade

```

pub struct DshaeCrystal {
    pub id: Uuid,
    pub surviving_triangles: Vec<(usize, usize, usize)>,
    pub net_edge_bps: Q32,
    pub execution_path: Vec<CandidateLeg>,
    pub stability_metric: Q32,    // persistence
    pub validity_ticks: u64,     // max age before expiry
    pub him_digest: Hash256,     // source HIM
    pub cascade_depth: u8,       // how deep the cascade went
}

impl DshaeCrystal {
    pub fn to_candidate_cycle(&self) -> CandidateCycle {
        // Convert DSHAE crystal to FSR CandidateCycle
    }
}

```

```

    // for the existing gate/CSP/execution pipeline
  }
}

```

2.3 DSHAE Configuration

Listing 7: DSHAE config section

```

dshae:
  enabled: true
  mode: "integrated"          # integrated | shadow
  basket:
    currencies: ["BTC", "ETH", "SOL", "USDT", "USDC"]
    min_basket_size: 3
    max_basket_size: 8
  dual_simplex:
    cycle_length: 3           # triangle = 3 legs
    notional_per_leg: 1000    # Q32: base notional
    anti_phase_tolerance: 100 # Q32: epsilon_phi
    rebalance_window: 5       # ticks to restore neutrality
  holographic:
    q32_scale: 1000000        # deviation tensor scaling
    min_points: 20            # minimum HIM points for cascade
  cascade:
    max_depth: 3
    dimensional_sequence: [5, 4, 3]
    dk_contraction_rate: 8500 # Q32: 0.85
    press_top_k: 16
  crystal:
    tau_edge_bp: 5            # minimum net-edge
    max_age_ticks: 3          # crystal validity window
    max_concurrent: 2         # max active crystals

```

3 Desktop GUI

3.1 Technology

- `egui 0.29+` (immediate-mode GUI library).
- `eframe` (egui native window backend).
- `egui_plot` (line charts, scatter plots).
- The GUI runs as the main thread. The engine runs on a background thread, pushing state updates via `mpsc::channel`.

3.2 New Crate: `fsr-gui`

Listing 8: `fsr-gui` module structure

```

fsr-gui/
src/
  lib.rs          // App struct, main loop
  app.rs          // eframe::App implementation
  state.rs        // GuiState (received from engine)
  panels/
    mod.rs

```

```

overview.rs      // main dashboard
pnl_chart.rs     // P&L time series chart
resonance.rs     // resonance metrics + SI chart
regime.rs        // regime history bar
crystals.rs      // crystal log + HIM scatter
ttcp.rs          // TTCP convergence chart
risk.rs          // risk panel (from Phase 3)
sniper.rs        // sniper status
config.rs        // config editor
sandbox.rs       // sandbox launcher + results
charts.rs        // egui_plot helpers
theme.rs         // color scheme, fonts

```

3.3 GUI Layout

The GUI uses a tab-based layout with a persistent top bar:

Listing 9: GUI structure

```

+==[ FIXPOINT SWARM-R ]====[ tick: 4821 | Alpha | Sniper: ARMED ]==+
|
| [Dashboard] [P&L] [Crystals] [TTCP] [Risk] [Config] [Sandbox]      |
|
| (active tab content fills the rest of the window)                    |
|
+-----+
| Status bar: PnL +23.7bp | DD -3.1bp | Crystals: 4 | chain: ok      |
+-----+

```

3.4 Tab Descriptions

Tab	Contents
Dashboard	All 7 FSM states, gate score gauge, resonance snapshot, candidate counts, sniper status. The “at-a-glance” screen. Similar to TUI but graphically richer.
P&L	Time-series line chart of cumulative net P&L (bp). Drawdown shaded area. Trade markers (green = profit, red = loss). Configurable time window.
Crystals	Table of recent DSHAE crystals with net-edge, triangle, stability. Scatter plot of HIM point cloud (2D PCA projection) colored by deviation intensity. Click a crystal to see its triangle details.
TTCP	Three convergence distance lines ($\delta_1, \delta_2, \delta_3$) over time. Crystal emission markers. Cascade depth gauge. Meta-Betti display.
Risk	Exposure, leverage, drawdown, inventory, daily loss limit progress bar. Same data as TUI risk panel but with graphical bars and gauges.
Config	View current config (read-only in running mode). Button to trigger hot-reload. Diff display after reload.
Sandbox	The validation sandbox (Section 4).

3.5 Engine-to-GUI Communication

Listing 10: State channel

```
pub struct GuiState {
    pub tick: u64,
    pub mode: RunMode,
    pub regime: RegimeState,
    pub fsm_states: FsmStates,          // all 7 FSMs
    pub gate_score: Q32,
    pub resonance: ResonanceSnapshot,
    pub candidates: CandidateSummary,
    pub pnl: PnlSummary,
    pub risk: RiskSummary,
    pub sniper: SniperStatus,
    pub ttcp: TtcpSummary,
    pub crystals: Vec<CrystalSummary>, // recent crystals
    pub him_projection: Option<Vec<f32; 2>>, // 2D PCA for scatter
    pub pnl_history: Vec<(u64, f32)>,    // tick -> cumulative PnL
    pub delta_history: Vec<(u64, f32, f32, f32)>, // tick -> d1, d2, d3
    pub events: Vec<DashboardEvent>,
}
```

The engine sends `GuiState` every tick via `mpsc::Sender<GuiState>`. The GUI reads the latest state on each frame (60 fps target). If multiple states arrive between frames, only the latest is rendered (drop stale).

GUI Rules

1. The GUI never modifies engine state directly. Config changes go through hot-reload.
2. If the GUI crashes, the engine continues running headless (same as TUI rule).
3. All chart data uses `f32` for rendering. Conversion from `Q32` happens at the GUI boundary, not in the engine.
4. The GUI must render at ≥ 30 fps even during heavy engine computation.

3.6 Launch

Listing 11: GUI launch

```
# Launch desktop GUI with paper mode
fsr gui --config balanced.yaml

# Launch desktop GUI with sandbox
fsr gui --sandbox

# Launch desktop GUI with historical replay
fsr gui --replay-historical data/recordings/session_*.rec

# Original TUI still works
fsr run-paper --config balanced.yaml # TUI mode (unchanged)
```

4 Validation Sandbox

The sandbox is the entry point for someone who has never used the system. One click, zero config, visual results.

4.1 Bundled Test Data

The repository ships with synthetic `.rec` files under `fixtures/sandbox/`:

File	Duration	Characteristics
<code>scenario_calm.rec</code>	5000 ticks	No arbitrage. All rates perfectly transitive. System should produce zero crystals, zero trades.
<code>scenario_arb_single.rec</code>	5000 ticks	One injected 10bp triangle mispricing at tick 2500, lasting 100 ticks. System should detect it.
<code>scenario_arb_recurring.rec</code>	10000 ticks	Five injected mispricings of varying intensity (3bp–15bp), spaced 1500 ticks apart. Tests sensitivity.
<code>scenario_noisy.rec</code>	10000 ticks	Many small (<2bp) random deviations but no real arbitrage. System should NOT trade. Tests false-positive resistance.
<code>scenario_regime_shift.rec</code>	10000 ticks	Calm → volatile → calm. Two real opportunities during volatile phase. Tests regime adaptation.

4.2 Synthetic Data Generator

Listing 12: Sandbox data generator

```
pub struct SandboxGenerator {
  pub basket: Vec<&'static str>, // ["BTC", "ETH", "SOL", "USDT"]
  pub base_rates: HashMap<(usize, usize), Q32>,
  pub seed: u64,
}

impl SandboxGenerator {
  /// Generate a .rec file with known arbitrage injections
  pub fn generate(
    &self,
    ticks: u64,
    injections: &[ArbitrageInjection],
    noise_level: Q32,
  ) -> Vec<MarketFrame> {
    // 1. Start with perfectly transitive rates
    // 2. Add Gaussian noise at noise_level
    // 3. At injection points, add specific triangle mispricings
    // 4. Return deterministic MarketFrame sequence
  }
}

pub struct ArbitrageInjection {
  pub start_tick: u64,
  pub duration_ticks: u64,
  pub triangle: (usize, usize, usize),
  pub magnitude_bps: Q32,
}
```

The generator runs at build time (`build.rs`) or via `fsr generate-sandbox`. Bundled `.rec` files are committed to the repo so the user never needs to generate them.

4.3 Sandbox GUI Tab

Listing 13: Sandbox tab UI

```

+--[ Sandbox - Validate the System ]-----+
|
| Choose a scenario:
| (*) Calm Market (no arb)      Expected: 0 trades, 0 crystals
| ( ) Single Arbitrage          Expected: 1 crystal near tick 2500
| ( ) Recurring Arbitrage       Expected: 3-5 crystals
| ( ) Noisy Market (no arb)     Expected: 0 trades
| ( ) Regime Shift              Expected: 2 crystals during volatility
|
| [>> Run Scenario <<]
|
| --- Results (after run) ---
| Status: PASS / FAIL
| Expected crystals: 1      Actual: 1      [OK]
| Expected trades: 1        Actual: 1      [OK]
| False positives: 0                [OK]
| Net P&L: +7.3 bp (expected: 5-12) [OK]
| Replay determinism: VERIFIED      [OK]
|
| [View Full Backtest Report] [View P&L Chart] [View Crystal Log]
+-----+

```

Each scenario has predefined **expected outcomes**: expected crystal count (range), expected trade count, expected P&L range, expected false positive count (always 0 for no-arb scenarios). After the run completes, the sandbox compares actual outcomes to expected and displays PASS/FAIL per criterion.

Sandbox Design Philosophy

The sandbox answers one question: “Does this system do what it claims?” The operator runs 5 scenarios, sees 5 PASS results, and has empirical evidence that: (a) the system does not trade when there is no opportunity, (b) the system detects real opportunities when they exist, (c) the system’s P&L matches expectations, (d) the system is deterministically reproducible. No domain knowledge required. No configuration needed. Just click and observe.

4.4 Scenario Validation Logic

Listing 14: SandboxScenario validation

```

pub struct SandboxScenario {
    pub name: String,
    pub recording_path: PathBuf,
    pub expected: ExpectedOutcome,
}

pub struct ExpectedOutcome {
    pub crystals_min: u64,
    pub crystals_max: u64,
    pub trades_min: u64,

```

```

pub trades_max: u64,
pub false_positives_max: u64, // always 0 for no-arb
pub pnl_min_bps: Q32,
pub pnl_max_bps: Q32,
pub invariant_violations: u64, // always 0
}

pub struct SandboxResult {
    pub scenario: String,
    pub passed: bool,
    pub actual_crystals: u64,
    pub actual_trades: u64,
    pub actual_pnl_bps: Q32,
    pub actual_false_positives: u64,
    pub replay_determinism: bool, // ran twice, same digests?
    pub backtest_report: BacktestReport,
    pub checks: Vec<(String, bool)>, // per-criterion pass/fail
}

```

5 Module Placement

Feature	Crate	Module
Basket, Simplex, Tensor, HIM, Crystal	fsr-dshae	New crate
GUI app, panels, charts	fsr-gui	New crate
Sandbox data generator	fsr-dshae	src/sandbox_gen.rs
Sandbox runner	fsr-gui	src/panels/sandbox.rs
DSHAE integration in macro-cycle	fsr-runtime	src/dshae_bridge.rs
DSHAE config	fsr-dshae	src/config.rs
Bundled scenario files	Repository	fixtures/sandbox/*.rec

6 New Dependencies

Crate	Version	Purpose
egui	≥ 0.29	Immediate-mode GUI rendering
eframe	≥ 0.29	Native window backend for egui
egui_plot	≥ 0.29	Line charts, scatter plots

All GUI dependencies are in `fsr-gui` only. `cargo build -workspace` without the `gui` feature must still work (headless build for CI and servers). The GUI binary is a separate target:

```

# In workspace Cargo.toml:
[workspace.members]
# ...
members = ["crates/fsr-gui"] # only built with --features gui

# Build headless (CI):

```

```
cargo build --workspace --exclude fsr-gui

# Build with GUI:
cargo build --workspace
```

7 Implementation Order

Step	Component	Done when
4.1	DSHAE types	fsr-dshae crate created. Basket, Tensor, Deviation Tensor, HimPoint, DshaeCrystal structs compile. Unit tests for tensor transitivity check.
4.2	Dual Simplex	Alpha/Beta cycle computation. Axle invariant enforcement. Phase anti-synchronization. Unit tests: Axle = 0 always, cycle product correct.
4.3	HIM construction	Build HIM from basket + rates + deviations. Q32 embedding. Digest computation. Unit test: N=4 basket produces $\binom{4}{3} = 4$ HIM points.
4.4	DSHAE cascade	DK/WT/Pi/Press applied to HIM. Crystal formation from surviving triangles. Net-edge computation. Tests: known arb → crystal, no-arb → no crystal.
4.5	FSR integration	dshae_bridge.rs in fsr-runtime . DSHAE replaces trumpet expansion when dshae.enabled = true . Paper mode works with DSHAE.
4.6	Sandbox generator	SandboxGenerator produces all 5 scenario .rec files. Files committed to fixtures/sandbox/ .
4.7	GUI shell	fsr-gui crate. Empty egui app launches, shows top bar and tab skeleton. fsr gui -sandbox opens window.
4.8	GUI Dashboard tab	FSM states, gate score, resonance snapshot. Reads from GuiState channel.
4.9	GUI P&L chart	Cumulative P&L line chart with drawdown shading and trade markers.
4.10	GUI Crystals tab	Crystal table + HIM scatter plot (2D PCA projection).
4.11	GUI TTCP tab	Three convergence lines + crystal markers over time.
4.12	GUI Sandbox tab	Scenario selector, run button, result display with PASS/FAIL. All 5 scenarios pass their expected outcomes.
4.13	Integration test	Full pipeline: fsr gui -sandbox runs all 5 scenarios, all pass. fsr run-paper -config balanced.yaml works with DSHAE enabled. All Phase 1–3 tests pass.

Step Closure Rule

Steps 4.1–4.6 are the DSHAE engine (no GUI needed). Steps 4.7–4.12 are the GUI (engine must work). Step 4.13 is the integration gate. Each step must compile and pass tests before the next begins.

8 Definition of Done (Phase 4)

Phase 4 is complete when:

1. All Phase 1–3 tests pass unchanged (176+ tests).
2. `cargo clippy -workspace -exclude fsr-gui - -D warnings` passes.
3. `fsr-dshae` crate compiles with unit tests for: tensor transitivity, axle invariant, HIM construction, crystal formation, no-arb quiescence.
4. `fsr run-paper -config balanced.yaml` with `dshae.enabled: true` runs 1000 ticks, produces crystals from synthetic data, P&L report is correct.
5. `fsr gui -sandbox` launches a desktop window with the sandbox tab.
6. All 5 sandbox scenarios produce correct PASS results.
7. Sandbox replays are deterministic (run twice, same digests).
8. GUI Dashboard, P&L, Crystals, and TTCP tabs render live data during paper-mode run.
9. GUI does not block the engine (engine continues if GUI is slow).
10. `fsr gui -config balanced.yaml` runs paper mode with live visual dashboard.
11. No network access required for any sandbox or paper-mode operation.

9 Phase 5 Preview (Non-Normative)

- Full persistent homology (Ripser Rust bindings) replacing β_0 -only TTCP.
- Strategy rotation: DSHAE + original trumpet as concurrent signal sources with weighted blending.
- Telegram/Signal alerts on crystal emission or safe-hold.
- Automated promotion: N consecutive successful backtests $\rightarrow$ auto-approve.
- Portfolio view: multiple concurrent configurations tracked in one GUI.

End of Phase 4 Specification